Policy Impact Assistant

Handbook for staff, administrators and engineers

Find every policy affected by a change to another, with the evidence for each, on your own hardware. This handbook covers how it works, daily use, installation, operations, security, review and evaluation.

secretary-keeper version 0.1 (langgraph branch), October 2026. MIT License.

Contents

1 **Architecture** 3

How it works and why it's built this way

For: Everyone

2 **How to use it** 15

Ask questions and read the results; add policies, groups and models

For: Staff and administrators

3 **Setup** 19

Install it on the GPU server

For: Engineers

4 **Operations** 25

Run it day to day, upgrade it, fix problems

For: Administrators and engineers

5 **Security and safety practices** 31

Keep policy text contained and the system safe

For: Everyone

6 **Review queue and judge training** 36

Let people check answers, then teach the judge from them

For: Administrators, reviewers and engineers

7 **Evaluation** 41

Measure whether it finds the right policies

For: Policy owners and engineers

8 **Testing** 44

Run and understand the tests

For: Engineers

Chapter 1

Architecture

How it works and why it's built this way.

For: Everyone

This is the **langgraph** version. People chat with Granite in Open WebUI. Granite calls a tool, and the tool asks policy-api to run the check. Inside policy-api, each check runs as a LangGraph graph that saves its state to Postgres after every step. Other versions of the product are described in the README.

The job

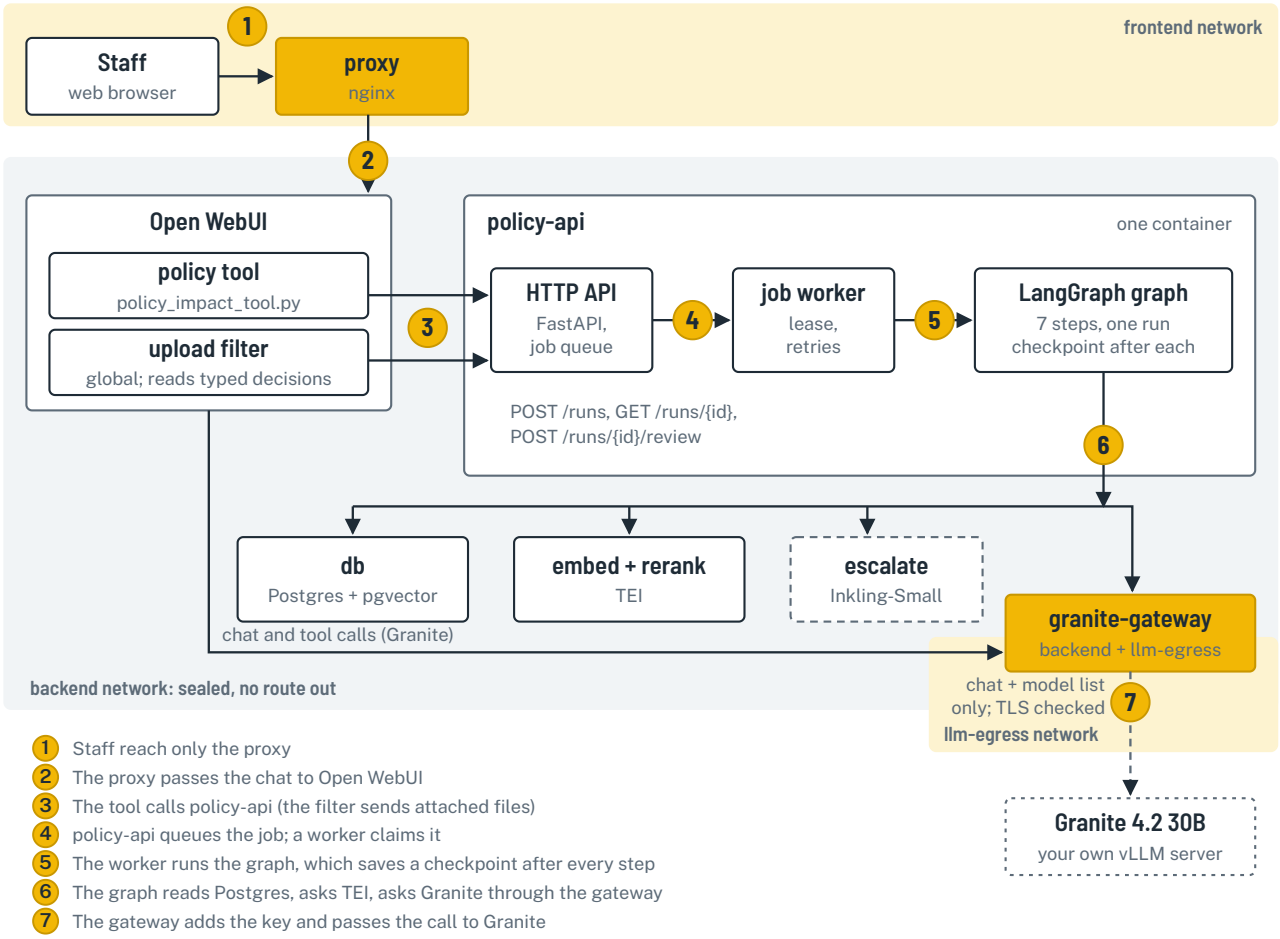
When a head policy changes, find every policy in a chosen group that now conflicts with it, repeats its old wording, points to a section that moved, or uses a defined term whose meaning changed. Missing an affected policy is the expensive mistake, so the design favours **recall** (finding everything) over precision (flagging nothing extra), and it puts a person in the loop for anything uncertain.

The core rule

The chat model never decides impact itself. It works out what the person is asking, calls the tool, and passes on the result. The graph inside policy-api makes every decision. The routing between steps is plain code, not a model's choice. Code builds every number, table and spreadsheet. The chat answer and the Excel file therefore always agree, and every decision can be traced in the audit trail.

With `REVIEW_GATE=1` there is a second rule: the chat model cannot settle a "maybe" either. Only the person's own typed message can (Review pause).

The parts in one picture



The request path. People reach only the proxy; policy-api and Open WebUI reach Granite only through the gateway.

Read it from left to right. People reach only the proxy. Everything that holds policy text sits on the sealed backend network. The only way out is `granite-gateway`, which reaches your Granite server over the `llm-egress` network. The numbers 1 to 7 mark the path of one check; the legend lists them.

Request flow, step by step

A. A question in the chat

Example: "Which Safety policies are affected by the change to POL-007?"

1. The person types in Open WebUI, reached through the `proxy`.
2. Open WebUI sends the chat to Granite through `granite-gateway`, with the system prompt (`openwebui/system_prompt.md`) and the tool list. Function calling is set to **Native**.
3. Granite replies with a tool call, for example `check_policy_change(policy_group, head_policy)`.

4. Open WebUI runs the tool (`openwebui/policy_impact_tool.py`). The tool calls policy-api with `POLICY_API_KEY`. If a group or policy name is not a clear match, the tool returns a question for the person instead of guessing.
5. The tool's result goes back to Granite, which shows it. The tool tells Granite to show answers exactly as written, so numbers, policy IDs and links reach the person unchanged.

The tools are `list_policy_groups`, `find_policy`, `check_policy_change`, `get_check_result`, `make_check_files`, `compare_policies` and `make_document`. There is **no search tool** in this version. A question about what a policy says ("What do we require for lockout devices?") has no tool behind it, and the system prompt tells Granite to say it does not know. The `hermes` branch adds search and policy-text tools.

B. A question with an attached document

1. The person attaches a file. Before the message leaves Open WebUI, the **upload filter** (`openwebui/upload_filter.py`, a global function) sends the file to `POST /uploads` on policy-api with `POLICY_API_KEY`. It reads only files that this person uploaded and that sit in Open WebUI's upload folder.
2. policy-api reads the file in its own limited process, splits it into numbered sections, stores it, and guesses which stored policy it is a new version of.
3. The filter adds a note to the end of the person's message: `[Attached file ...: upload id ...]`. It also stops Open WebUI from cutting the file into its own search index.
4. Granite sees the note and passes the upload id to `check_policy_change`.

C. A policy change check, end to end

1. The tool finds the group (`GET /subsets`) and the head policy (`GET /policies`), then sends `POST /runs` with the asking person's email or name as `requested_by`. policy-api stores a `runs` row and a row in the `jobs` table, and answers **202 Accepted** with a run id.
2. A worker inside policy-api claims the job (`FOR UPDATE SKIP LOCKED`) with a 60-second lease and renews the lease every 20 seconds while it works. The run's status becomes `running`.
3. The worker starts the graph with the run id as the **thread id**. The graph runs seven kinds of step (The graph). After each step, LangGraph writes a checkpoint to Postgres and waits for the write to finish before the next step starts (`durability="sync"`).
4. While this runs, the tool asks `GET /runs/{id}` every 3 seconds (`POLL_SECONDS`) and shows the `progress` text in the chat. The tool gives up waiting after `MAX_WAIT_MINUTES` (30), but the check carries on.
5. If the graph finishes, the worker records `done` and the result. `GET /runs/{id}` then returns the chat answer (built in code) and a signed Excel link that works for 72 hours.
6. If the graph reaches the review step with `REVIEW_GATE=1`, the check pauses. See Review pause.
7. If the person then asks for a PDF, Word file or charts, Granite calls `make_check_files`.

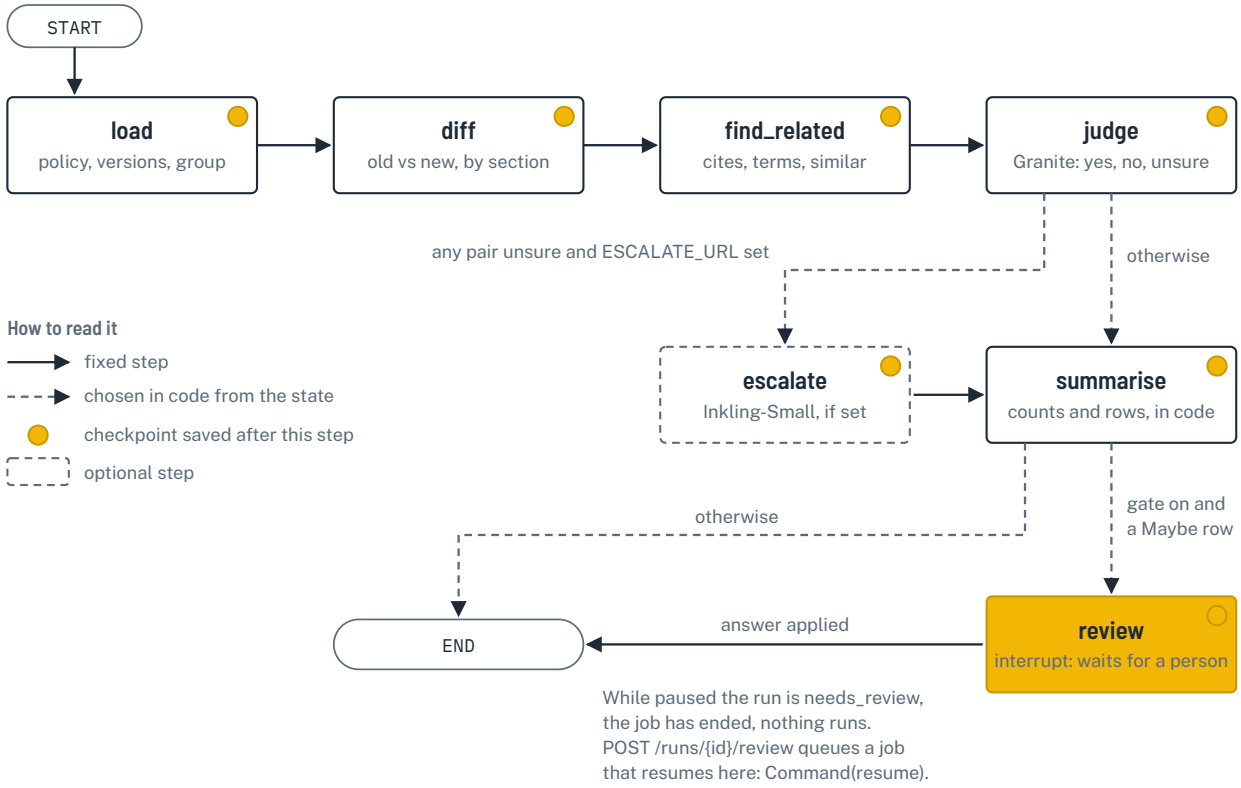
Retries. A step that fails for a temporary reason (database blip, model restart) raises an error. The worker waits (random, growing up to 5 minutes) and tries the job again, up to `JOB_MAX_ATTEMPTS` (3). The retry **carries on from the last saved checkpoint**; it does not start over. A problem with the question itself (no such policy, nothing changed) is a `RunError`: it is shown to the person at once and is not retried.

Crash and resume. If policy-api is stopped or dies mid-check, the job's lease runs out (or, after a restart, the worker takes its own jobs back at once). Any worker claims the job again. The worker asks LangGraph for the thread's saved state. If steps are left, it continues from the first unfinished one. The people waiting in Open WebUI see "Starting again after an interruption…". Work inside one step is repeated: for example, a half-finished `judge` step asks its pairs again. The step first deletes the verdicts it had already saved for its stage, so the audit trail never holds duplicates.

D. Deciding "maybe" sections (`REVIEW_GATE=1`)

1. The graph reaches `review`. It calls `interrupt()`, which saves a checkpoint and stops. The worker sets the run to `needs_review` and finishes the job. Nothing is running or queued.
2. The tool sees `needs_review`, shows the list of "maybe" sections and tells the person what to type (see the message format below).
3. The person types a message that starts with `decide`: `decide 3f9a1c2e SAF-150 3.1 yes, SAF-151 2.4 no`. The 8 characters are the **check number**. `decide 3f9a1c2e all no` answers every listed section at once.
4. The upload filter reads this from the person's own message only, never from an assistant or tool message. It checks the format and sends `POST /runs/{id}/review` with the person's Open WebUI name as `reviewed_by`.
5. policy-api checks that each section is on the waiting list, then **claims** the check with one atomic update (`needs_review` to `queued`). Two answers sent together cannot both apply. It queues a `review_run` job.
6. A worker takes that job and resumes the graph with `Command(resume=answer)`. The `review` step turns each "yes" into a confirmed "Yes" row and drops each "no". Sections the person left out stay "Maybe". The check ends as `done`.
7. The filter adds a note to the person's message ("decisions were recorded"). Granite then calls `get_check_result` to show the final answer.

The graph



One check as a graph. Code chooses every branch; the state is saved to Postgres after every step.

The graph is in `policy_api/app/graph.py`. It is built with LangGraph's `StateGraph`. The two model steps ask about one pair at a time; every other step is ordinary code.

Solid arrows are fixed steps. The two labelled branches are chosen by small functions in code, from the state alone. After every step, the whole state is saved to Postgres.

The state

The state (`RunState`) holds plain JSON values only, so a saved checkpoint never depends on our class layout. Each step returns only the keys it changes.

Key	Holds	Set by
`run_id`	The check's id. Also the LangGraph thread id	Input
`head`	Head policy id and title	`load`
`labels`	Names of the two versions compared ("version 3", "uploaded file ...")	`load`
`policy_ids`, `metas`	Policies in the group (without the head policy) and their titles and owners	`load`

Key	Holds	Set by
`changes`	Each changed section, with its saved `changes` row id	`diff`
`pairs`	Each (changed section, related section) pair: the row, how it was found, the verdict, and whether Inkling-Small saw it	`find_related`, then `judge`, then `escalate`
`errors`	How many pairs the model could not answer	`judge`
`result`	The finished answer: counts, rows, not-affected list, method. Chat text and Excel both come from it	`summarise`, then `review`
`reviewed`	The person's decisions	`review`

The steps

Step	What it does	Reads	Writes
`load`	Finds the head policy and its two versions (or the stored version and the upload). Lists the group's other policies. Fails with a plain message if something is missing	`runs`, `policies`, `uploads`	`head`, `labels`, `policy_ids`, `metas`
`diff`	Clears any earlier attempt's rows, compares old and new text section by section (in a thread, with a time cap), saves each change	`policy_versions`, `uploads`	`changes`; `changes` table
`find_related`	For each change, collects sections in the group that cite the head policy, use a changed defined term (both always kept), or are similar in meaning (embedding search, then reranker keeps the top N)	`changes`, `refs`, `clauses`, TEI	`pairs`
`judge`	Granite reads each pair and answers yes, no or unsure, with a reason, quotes and a suggested edit, forced into a JSON schema. Up to `JUDGE_CONCURRENCY` (16) at once. Each verdict is saved. A pair the model cannot answer becomes "unsure" with a note	`pairs`	`pairs`, `errors`; `judgements` table
`escalate`	Optional. Sends only the "unsure" pairs to Inkling-Small for a second opinion. If it fails, the first opinion stays	`pairs`	`pairs`; `judgements` table
`summarise`	Per policy: Yes if any section is "yes", Maybe if any is "unsure", otherwise not affected. Pure code	all of the above	`result`
`review`	Only with `REVIEW_GATE=1` and at least one Maybe row. Pauses until a person answers, then applies the answer	`result`	`result`, `reviewed`

The branches

After	Goes to	When
`judge`	`escalate`	Any pair is "unsure" **and** `ESCALATE_URL` is set
`judge`	`summarise`	Otherwise
`summarise`	`review`	`REVIEW_GATE` is on **and** the result has at least one Maybe row
`summarise`	end	Otherwise

The other connections are fixed: `load` to `diff` to `find_related` to `judge`, `escalate` to `summarise`, and `review` to end. The path through the graph is a straight line with two optional detours. It has no loops, so it cannot run on forever.

Recursion limit. Every run is started with `recursion_limit=25`. The graph has at most seven steps per run. A run that came near 25 would be a bug, and LangGraph would stop it with an error rather than let it spin.

In `find_related`, the sections that cite the head policy or use a changed term are never cut by the reranker, because explicit links are where most real impact is. The similar-meaning search catches policies that overlap in meaning without citing the head policy.

Checkpoints

	Production	Demo mode and most tests
Checkpointer	`AsyncPostgresSaver` (`langgraph-checkpoint-postgres` 3.2.0), using `DATABASE_URL`	`InMemorySaver`
Where saved	Tables in the same Postgres database as the policies	In the policy-api process
Survives a restart	Yes: a check resumes, and a paused check stays paused	No: a restart loses the state
Tables	Created on every start by `saver.setup()` (safe to repeat)	None

`open_checkpointer` picks Postgres whenever `DEMO_MODE` is off and `DATABASE_URL` is set. The tests that use `TEST_DATABASE_URL` use Postgres too (testing.md).

What is stored where

Store	Tables	Holds	Who writes it
Checkpoints	`checkpoints`, `checkpoint_blobs`, `checkpoint_writes`, `checkpoint_migrations`	The graph's state after each step, per thread (one thread per check). This is how a check resumes	LangGraph
Application	`runs`, `changes`, `judgements`, `uploads`	The check's status, progress, final result; each change; every model verdict with its exact prompt (the audit trail); uploaded files	policy-api
Job queue	`jobs`	`impact_run` and `review_run` jobs with lease, attempts and last error	policy-api
Policies	`policies`, `policy_versions`, `clauses`, `refs`, `subsets`	The policy library and its reference graph	`app.ingest`

The checkpoint tables are for **resuming**. The application tables are for **reading and auditing**. The status the chat shows always comes from `runs`, not from the checkpoints. Checkpoints are not cleaned up automatically; they are small, and keeping them means a finished check's steps can be inspected. To purge them with old runs, see security.md.

Review pause (`REVIEW_GATE`)

`REVIEW_GATE` is off by default. When it is off, "maybe" results go into the answer and the Excel file, and people decide them afterwards (the Review decision column, or the review queue). When it is on, a check that has "maybe" rows stops at `review` and waits.

What the API shows.

Route	Use	Notes
`GET /runs/{id}`	Status, progress and error. While `status` is `needs_review` it also returns `review`, a list of `policy_id`, `policy_title`, `clause`, `clause_heading` and `why` for each waiting section. When `done`, the answer and Excel link	Accepts the full id **or** the 8-character check number
`POST /runs/{id}/review`	Body: `decisions` (a list of `policy_id`, `clause`, `affected`) and optional `reviewed_by`. Answers 202 and `queued`	Accepts the full id or the 8-character check number. For a short number, only paused checks are searched

Every other route needs the full id. The answers to a bad review request are plain: 404 for no such check, 409 if the check is not waiting (or was just answered by someone else, or two checks match a short number), 422 if a section is not on the waiting list.

The check number is the first 8 hex characters of the run id, for example `3f9a1c2e`. It is only 32 bits and is not tied to the person who asked (Limits).

Partial answers. The person can decide some sections and leave the rest. The check then finishes with the undecided ones still marked Maybe. There is no second pause.

Never answered. A paused check waits for ever. No timer closes it. It uses no worker slot and no model. Its state is safe in Postgres, so it survives restarts. Ask the assistant for the check number and the list is shown again (operations.md).

Components

Component	Image and version	Role	Networks
`proxy`	`nginx:1.27-alpine` (`NGINX_TAG`)	The only thing people reach: Open WebUI, plus report downloads and `/health`. Holds no data	backend + frontend
`open-webui`	`ghcr.io/open-webui/open-webui` (`OPENWEBUI_TAG`, pinned in `.env`)	Chat screen. Sends chats to Granite. Runs the tool and the upload filter. Trimmed: no web search, memories or code running	backend
`policy-api`	Built from `policy_api/` (Python 3.12, FastAPI 0.115 or later, LangGraph 1.2.16, `APP_VERSION`)	The HTTP API, the job queue and worker, the graph, uploads, reports and downloads	backend
`db`	`pgvector/pgvector:pg17` (`PGVECTOR_TAG`)	Postgres with pgvector: policies, clauses, graph, runs, jobs, audit trail, and the checkpoint tables	backend

Component	Image and version	Role	Networks
`embed` , `rerank`	`text-embeddings-inference` (`TEI_TAG`) with Granite embedding and reranker models	Embeddings for similarity search; reranker to order and cut candidates	backend
`granite-gateway`	`nginx:1.27-alpine`	The backend's only way out. Passes chat and model-list calls to `GRANITE_URL` , adds the Granite key, checks TLS	backend + llm-egress
Granite 4.2 30B	Your own vLLM server, outside this stack (vLLM 0.20 or later)	The chat model, and the judge for every pair	Reached only through the gateway
`escalate` (optional)	`vllm/vllm-openai` (`VLLM_TAG`), Inkling-Small	Second opinion on "unsure" pairs	backend
`extract` (optional)	`vllm/vllm-openai` (`VLLM_TAG`), gpt-oss-20b	Finds references the regex misses during ingest	backend
Review and training services (optional)	Argilla v2.8.0 and its helpers; `lora-trainer` ; `judge-candidate` ; `judge-tuned`	Review queue and fine-tuning (review-and-training.md)	backend

The graph runs **inside** `policy-api` . There is no separate graph or agent container. The tool and the upload filter run inside Open WebUI, from files pasted into its admin screens (setup.md).

Network and security boundaries

```
frontend network:   people → proxy
backend network:    proxy → open-webui → policy-api → db, embed, rerank
backend + egress:   open-webui and policy-api → granite-gateway → Granite server
```

- `backend` is `internal: true` : no route out. `proxy` is the only container on `frontend` . The gateway is the only container on `llm-egress` .
- The gateway passes `/v1/chat/completions` and `/v1/models` and nothing else, and checks the Granite server's certificate.
- policy-api answers only requests with the right key and a known `Host` name. The proxy shows people only Open WebUI, report downloads and `/health` .
- The checkpoints are in the same database as the policies and add no new network path.

Four keys keep the parts apart. Each is held by as few parts as possible.

Key	Where it lives	Who uses it	What it protects
`POLICY_API_KEY`	`.env`	policy-api (to check it); the tool and the upload filter (a setting, called a valve, in Open WebUI)	Every policy-api route except signed downloads
`GRANITE_API_KEY`	`.env`	`granite-gateway` only	The Granite server. Open WebUI and policy-api never see it
`LINK_SIGNING_KEY`	`.env`	policy-api	Download links

Key	Where it lives	Who uses it	What it protects
`LLM_API_KEY`	`.env`	policy-api and the optional local models	The optional `escalate`, `extract` and trained-judge servers

See security.md.

When things fail

What fails	What happens
Granite server down or slow	Chat: Open WebUI shows an error. Check: each pair that fails is marked "could not be checked automatically" and shows as Maybe. If **every** pair fails, the check stops with "The checking model is not responding" and is not retried. Run it again later
Worker or policy-api crashes mid-graph	The lease runs out (or the restarted worker takes its job back). The job runs again and the graph resumes after the last saved step. The person sees "Starting again after an interruption…"
A temporary error inside a step (database blip, TEI restart)	The job is retried up to `JOB_MAX_ATTEMPTS` (3) with growing waits, each time from the last checkpoint
The job fails every time	After the last attempt the job is `dead` and the run is `failed`. The person sees "failed several times in a row". The error stays in `jobs.last_error` (operations.md)
A problem with the question (no such policy, nothing changed)	Not retried. The person gets a plain reason
Review never answered	The check stays `needs_review` indefinitely. No timeout, no worker used. Anyone with the check number can still answer it later
Two answers to the same review at once	The first claims the check. The second gets "not waiting for a decision" (409)
policy-api down	The tool says the service did not respond. A queued check waits in the database
Postgres down	Everything stops, including checkpointing. Work resumes when it is back
Checkpoint tables lost (restored from an older backup)	A paused or half-finished check cannot resume. Start it again. `runs` and `judgements` are unaffected
A check takes longer than 30 minutes of waiting	The tool stops waiting and tells Granite the check is still running. The check itself continues. The person asks again with the check number
A file cannot be read	The upload filter puts a plain note in the message; Granite tells the person

Limits

- **Check numbers are not tied to the requester.** Anyone who has seen a check number can decide that check, from any chat. The decision is recorded under the Open WebUI name of whoever typed it, so it can be traced. See security.md.
- **Everyone with an account can ask about every group.** There is no per-user group filter.

- **Training still targets gpt-oss-20b.** The fine-tuning scripts and the `judge-candidate` and `judge-tuned` services are set up for that model, not for Granite. Verdicts can be sent to a tuned judge; the chat stays on Granite (review-and-training.md).
- **Decisions in chat are not used for training.** They change the final answer. They are not copied to `judgements`. The review queue is the source of training data.
- **No search or policy-text tool.** This version answers only about checks, comparisons and files.
- **The Granite server must be set up for this.** Tool calling, structured output and a context of at least 32768 tokens (setup.md).
- **A very long check holds the chat open.** The tool waits up to 30 minutes (`MAX_WAIT_MINUTES`). Open WebUI, or a proxy you run in front of the stack, may give up sooner. The check carries on either way, and the result is fetched later by check number.
- **The chat model follows its rules by instruction, not by force.** It can word an answer badly. That is why numbers, links and verdicts are produced by code and passed through unchanged, and why only the filter, never the model, can record a decision.

Why these choices

Choice	Why	Instead of
A graph with fixed routing, run by code	The steps never change. Routing is deterministic, testable and auditable	An agent that plans its own steps (more tokens, results vary between runs, harder to audit)
LangGraph with Postgres checkpoints	A crash resumes after the last finished step instead of repeating a long, costly run. Pausing for a person is built in (`interrupt`)	A hand-written state table and pause logic
The graph inside policy-api	No new container to run, secure or back up. Checkpoints live in the database that is already backed up	A separate workflow server (Temporal, Dagster): another stateful system inside the contained network
Review pause as a graph interrupt	The paused check holds no process and no model. It survives restarts	Keeping a worker waiting, or a custom "waiting" status with its own resume code
Decisions typed by the person, read by the filter	A model cannot approve its own "maybe". Prompt injection in a policy cannot record a decision	A tool the model calls to record a decision
Granite 4.2 30B as judge and chat model	US-developed (IBM), Apache 2.0, dense 30B that fits one 80 GB GPU in bf16, 128K context, tool calling, thinking can be switched off per request for fast, repeatable verdicts	Inkling for everything (needs 4–8 high-end GPUs per replica)
Use the Granite server you already run, behind one gateway	No second copy of a 58 GB model to host and update. One narrow, checked way out (two routes, one address, key held in one place) instead of a model container inside the stack	Starting a local judge container, or letting each service call the server on its own
Inkling-Small only for "unsure"	Larger-model judgement where it matters, at a fraction of the cost	Running every pair through the large model

Choice	Why	Instead of
Reranker before judging	Cuts hundreds of similar clauses to about 20 per change; the biggest compute saving	Judging every similar clause
JSON-schema guided decoding	Every verdict is valid and machine-readable	Asking for JSON in the prompt and hoping
Postgres for the graph of references	Recursive queries cover policy hierarchies; one fewer system to run	A separate graph database
Excel built with openpyxl	Exact, repeatable, safe from formula injection	Letting the model write the spreadsheet
Job queue in Postgres (`FOR UPDATE SKIP LOCKED`, leases)	Runs survive restarts with no new service to run or secure; Postgres is already here	Temporal or Dagster, for a queue of a few hundred jobs a day

Data model (db/init.sql)

- `policies`, `policy_versions`, `clauses`: every version is kept. `current_clauses` is a view of each policy's latest version.
- `refs`: which clause cites which policy and section. `subsets`: which policies are in which group.
- `uploads`, `runs`, `changes`: each question and what changed.
- `judgements`: one row per model decision, with model name, the exact prompt and its hash, verdict, and the reviewer's decision. This is the audit trail and the training data.
- `jobs`: the work queue (see operations.md). `schema_migrations` records which files in `policy_api/app/migrations/` have been applied; `db/init.sql` is the starting point for a new database.
- `checkpoint_*` tables: made by LangGraph on start, not by `db/init.sql`.

Clause parsing rules

A new clause starts at a line that begins with a section number:

- a Markdown heading with a number (`## 4 Lockout devices`)
- any multi-level number (`4.2 Devices are inspected…`, `4.2.1) …`)
- a single number followed by a short title (`4 Lockout devices`)

`1. Do this.` inside a clause stays part of that clause. Text before the first number is clause `0`. Policy IDs are found with `POLICY_ID_PATTERN` (default `[A-Z]{2,6}-\d{1,5}`), in forms like `POL-007 §4.2`, `§4.2 of POL-007`, `POL-007 section 4` and `POL-007` alone (whole policy).

Roadmap

Built since v0.1: the job queue, the review queue, LoRA fine-tuning, reports and charts, and in this version the LangGraph run with checkpoints and the review pause.

Next: OCR for scanned PDFs and Word's automatic numbering (known limits), and Kubernetes manifests for models that need more than one node.

Chapter 2

How to use it

Ask questions and read the results; add policies, groups and models.

For: Staff and administrators

For staff

Ask a question

Open the Policy Impact Assistant and type your question in your own words. Name two things:

1. **The group of policies to check**, such as Safety or HR
2. **The policy that changed**, either by its ID or name, or by attaching the new version

Examples:

> Which Safety policies are affected by the change to POL-007?
>
> Here's the new lockout policy. What in Safety needs updating? *(attach the file)*
>
> Check the HR group against the latest version of the leave policy.

If something is unclear, the assistant asks you one short question. A check usually takes one to a few minutes. You'll see progress messages while it works.

Attaching a file: use `.docx`, `.pdf`, `.md` or `.txt`. The sections must be numbered (1, 2.1, 4.3…), because the system compares the policy section by section.

Without a file, the assistant compares the two newest stored versions of the policy. If only one version is stored, it asks you to attach the new one.

Read the answer

> **2 of the 5 Safety policies need updating because of the change to POL-007 (Lockout/Tagout).**

Then you'll see:

- **What changed** in the head policy, section by section
- **A table** of the sections that need updating, and why
- **"Might be affected"** policies. A person must check these: the system wasn't sure.
- How many were checked and are **not affected**
- **A link to the Excel file** with all the detail

Decide the "maybe" sections

Your administrator may have turned on the review pause. Then a check that has "maybe" sections does not give its final answer straight away. The assistant lists the sections and a **check number** (8 letters and digits, such as `3f9a1c2e`).

Read each section and decide: is it affected by the change? **yes** means it needs updating. **no** means it does not. Then type a message that **starts with the word decide**, then the check number, then each section with your answer:

```
decide 3f9a1c2e SAF-150 3.1 yes, SAF-151 2.4 no
```

To give one answer to everything on the list, type `decide 3f9a1c2e all no` (or `all yes`). The assistant tells you it was recorded and shows the final answer. Your name is recorded next to your decisions.

Good to know:

- Type it yourself. The assistant cannot decide for you, and it will say so.
- The message must **start** with `decide`. A line pasted further down a message is ignored.
- You can decide some sections now and leave the rest. Those stay "might be affected".
- If you type something the system cannot read, nothing is recorded and the assistant shows you how to write it again.
- Do not share check numbers outside the team. Anyone who has one can decide that check.

Use the Excel file

Sheet	What's on it
Summary	The question, the answer and the counts
Impacted	One row per affected section. **Red = needs updating. Yellow = a person must decide.** Each row shows the old and new wording, what the section says now, why it's affected, quotes as evidence, and a suggested edit
Head policy changes	Every changed section, old wording beside new
Checked, not impacted	Every other policy in the group, and why it was ruled out
Method	Which models and settings were used (for audits)
Charts	A bar chart of affected sections by policy
Change matrix	Which change in the head policy affects which policy

Fill in **Review decision** (Agree, Disagree, Not sure) and **Reviewer** as you go. The file is yours to keep and share inside the organisation.

The download link stops working after 72 hours. Ask the assistant again for a new one.

Get a report, chart or comparison

Just ask, in your own words, after a check or on its own:

Say something like	You get
"Can I have that as a PDF?" / "...as a Word report?"	A report of the last check: the answer, charts, the table of affected sections, and the old and new wording with changes marked
"Show me a chart of that"	Charts in the chat: affected sections by policy, which change affects which policy, kinds of impact
"Compare POL-007 version 3 with version 4"	A side-by-side comparison, section by section. Removed words are red and crossed out; new words are blue and underlined
"Compare this file with POL-007" (with a file attached)	The same comparison, against your file
"Compare SAF-112 and SAF-130 in Excel"	One row per section, in a workbook
"Make a one-page Word summary of this for the safety committee"	A document the assistant writes from what's in the chat. It says so at the top: check it before relying on it

Files can be Word (.docx), PDF, or Excel (.xlsx). Links last 72 hours; one-off documents are kept for 30 days, and reports of a check can always be made again.

What the system can and can't do

- It **finds and explains.** People still decide and approve every policy change.
- **"Maybe" never means "fine".** Always check these.
- It only knows the policies that have been loaded. If one seems missing, tell the system owner.
- It can be wrong in both directions. If it flags something that isn't affected, or misses something that is, tell the system owner. Mistakes are how it gets better.

Known limits

- **Scanned PDFs** (pictures of pages) can't be read yet. Use the Word or text version.
- **Word's automatic numbering** isn't stored as text, so a Word file whose numbers are automatic may appear to have no sections. Save it with the numbers typed in, or as PDF.
- **Policies outside the group** you named are not checked. Ask about each group separately.

For admins

Add or update policies

Put the files in `POLICY_SOURCE_DIR` and run:

```
docker compose run --rm policy-api python -m app.ingest /policies
```

Ingest is safe to repeat. A file whose header has the same `version` replaces that version; a higher `version`, or no version, adds a new one. Earlier versions are kept, so "compare the two newest versions" works without uploading anything.

Add a group, or move a policy between groups

Edit `subsets:` in the policy's header, then ingest that file again. A policy can be in several groups. A policy's groups are replaced each time it's ingested.

Turn on the second-opinion model (Inkling-Small)

Start it with `docker compose --profile escalate up -d escalate`, set `ESCALATE_URL=http://escalate:8000/v1` in `.env`, then run `docker compose up -d policy-api`. "Unsure" pairs then go to Inkling-Small. The **Checked by larger model** column in Excel shows which.

Find more references when ingesting

`--llm-refs` asks the `extract` model to find references that the ID pattern misses, such as "the lockout policy". Start the model first (`--profile extract`) and set `EXTRACT_URL` .

Change how wide the search is

In `.env` :

- `SEMANTIC_K` (default 30): how many similar clauses to consider for each change
- `RERANK_KEEP` (default 20): how many of those to judge

Raise them if evaluation shows misses on policies that don't cite the head policy. Lower them if runs are too slow. Clauses that cite the head policy, or use a changed term, are always judged regardless of these settings.

Look up what happened in a run

Every decision is in the `judgements` table:

```
docker compose exec db psql -U policy -c \
  "SELECT candidate_policy, candidate_number, stage, model, result->>'impacted', result->>'why'
   FROM judgements WHERE run_id = '<run id from the Excel Summary sheet>' ORDER BY
candidate_policy"
```

Chapter 3

Setup

Install it on the GPU server.

For: Engineers

This guide installs secretary-keeper on one GPU server with Docker Compose. Follow the steps in order. Each step ends with a check; don't move on until it passes.

1. What you need

Item	Minimum	Notes
Granite server	An existing vLLM server running Granite 4.2 30B (about 58 GB in bf16, so one 80 GB GPU or two 48 GB)	Not part of this stack. See the next section
GPUs for this stack	1 smaller GPU for `embed` + `rerank`	Optional: 1 for `extract`, and 4 Blackwell or 8 Hopper for `escalate`
NVIDIA driver + Container Toolkit	Recent enough for your vLLM image	`docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi` must list your GPUs
Docker Engine + Compose v2	Compose 2.20 or later	`docker compose version`
Disk	About 400 GB	Mostly model weights
A machine with internet access	Temporary	To download images and weights before moving them in

Granite server requirements

The main model, IBM Granite 4.2 30B (Apache 2.0, dense, about 29B parameters, 128K context), is **not** started by this stack. It already runs on your own vLLM server. This stack calls it. Ask whoever runs that server to confirm each line:

Setting on the Granite server	Value	Why
vLLM version	0.20 or later	Earlier versions do not support Granite 4.2
Model name	`--served-model-name granite-4.2-30b`	Or keep your own name and set `GRANITE_MODEL` in `.env` to match
Precision	`--dtype bfloat16`	About 58 GB of weights
Tool calling	`--enable-auto-tool-choice --tool-call-parser qwen3_coder`	The chat model calls the policy tools

Setting on the Granite server	Value	Why
Reasoning	`--reasoning-parser nemotron_v3`	Built into vLLM. The model card's `granite_thinking_parser.py` plugin also works, but it runs code from the model download, so it is optional
Context length	`--max-model-len` of at least 32768	Enough for this version. Larger is fine; 131072 is what the model supports. (The `hermes` branches need 65536 or more)
Structured output	JSON-schema responses must work (vLLM guided decoding)	Every verdict is requested with `response_format` `json_schema`
API key	`--api-key <key>` (recommended)	Put the same key in `GRANITE_API_KEY`
Address	An `https://` address, unless the server runs on this same machine	Put it in `GRANITE_URL`, without `/v1`

Quick check, from a machine that can reach the server:

```
curl -s -H "Authorization: Bearer $GRANITE_API_KEY" $GRANITE_URL/v1/models
```

Check: the answer lists `granite-4.2-30b` (or the name you set in `GRANITE_MODEL`). Leave out the `-H` part if the server has no key.

The gateway

Nothing in the stack talks to the Granite server directly. The `granite-gateway` container (nginx, `gateway/granite.conf.template`) is the only container on the `llm-egress` network, and the only way out of the sealed backend network. It:

- passes on only `/v1/chat/completions` and `/v1/models`, and returns 404 for everything else;
- adds `GRANITE_API_KEY` itself and drops any key the caller sent, so the key lives in one place;
- checks the server's certificate. For a certificate from a private CA, put the CA file in `gateway/` (for example `gateway/granite-ca.pem`) and set `GRANITE_CA_PATH=/etc/nginx/granite/granite-ca.pem` in `.env`. `*.pem` files in `gateway/` are not committed to git.

`JUDGE_THINKING=1` makes Granite reason before each verdict. It is slower and sometimes better. The default is off. Try it on your evaluation cases first.

2. Bring images and weights into the environment

On a machine **with** internet access:

```
# Images: pin the exact tags you will write into .env
docker pull vllm/vllm-openai:<VLLM_TAG>
docker pull ghcr.io/huggingface/text-embeddings-inference:<TEI_TAG>
docker pull ghcr.io/open-webui/open-webui:<OPENWEBUI_TAG>
docker pull pgvector/pgvector:pg17
docker pull nginx:1.27-alpine
# Tag must equal APP_VERSION in .env, so Compose uses this image instead of trying to build one.
```

```
docker build -t secretary-keeper/policy-api:0.1.0 policy_api    # needs PyPI or your internal
mirror
docker save -o secretary-keeper-images.tar vllm/vllm-openai:<VLLM_TAG> ... secretary-
keeper/policy-api:0.1.0

# Weights (see models/README.md for the list)
hf download ibm-granite/granite-embedding-english-r2 --local-dir models/embedding
hf download ibm-granite/granite-embedding-reranker-english-r2 --local-dir models/reranker
# Granite itself stays on your vLLM server, so it is not copied here.
scripts/verify_models.sh record
```

Move the tar file, the repo and the `models/` folder into the environment. Then:

```
docker load -i secretary-keeper-images.tar
scripts/verify_models.sh check
```

Check: `verify_models.sh check` prints "All model files match their recorded hashes."

3. Configure

```
cp .env.example .env
for k in POSTGRES_PASSWORD POLICY_API_KEY LINK_SIGNING_KEY LLM_API_KEY OPENWEBUI_SECRET_KEY; do
  sed -i "s/^$k=.*/$k=$(openssl rand -hex 32)/" .env
done
chmod 600 .env
```

Then edit `.env`:

- `GRANITE_URL`, `GRANITE_API_KEY`, `GRANITE_MODEL`: your Granite server (see above). Add `GRANITE_CA_PATH` if its certificate comes from a private CA.
- `VLLM_TAG`, `TEI_TAG`, `OPENWEBUI_TAG`: the exact tags you loaded.
- `PUBLIC_BASE_URL`: the address people's browsers use for downloads, e.g. `https://policy.plant.local:8100`.
- `POLICY_ID_PATTERN`: if your IDs don't look like `ABC-123`.
- `POLICY_SOURCE_DIR`: the folder holding your policy files.

Assign GPUs to the models this stack runs itself (embedding, reranker and any optional ones). Granite is not on this list. `compose.yaml` names no GPUs, so you choose them for this server:

```
python3 scripts/setup_gpus.py
```

It lists your GPUs, asks which ones each model uses, checks the plan (for example that no two big models share a GPU), writes `compose.gpu.yaml` and sets `COMPOSE_FILE`, `ESCALATE_TP` and the other size settings in `.env`. Run it again whenever the hardware changes. To do it by hand instead, follow the comments in `compose.gpu.example.yaml`, which also has sizing notes per model.

If your embedding model's output size is not 768, change `vector(768)` in `db/init.sql` now, before the database is created.

Check: `docker compose config --quiet` prints nothing.

4. Start the stack

```
docker compose up -d db embed rerank granite-gateway
docker compose logs -f embed rerank   # wait until both say they are ready
docker compose up -d policy-api open-webui proxy
```

Optional models:

```
docker compose --profile escalate up -d escalate   # then set
ESCALATE_URL=http://escalate:8000/v1 in .env
docker compose --profile extract up -d extract     # then set EXTRACT_URL=http://extract:8000/v1
in .env
docker compose up -d policy-api                     # restart to pick up .env changes
```

On its first start, policy-api creates the LangGraph **checkpoint tables** (`checkpoints`, `checkpoint_blobs`, `checkpoint_writes`, `checkpoint_migrations`) in the same Postgres database. You do not run anything for this, and it is safe to repeat on every start. It needs `DEMO_MODE=0` and a `DATABASE_URL`; the Compose file sets both.

Check: `docker compose ps` shows every service as running or healthy. To prove the gateway reaches Granite: `docker compose exec policy-api python3 -c "import urllib.request as u; print(u.urlopen('http://granite-gateway:8000/v1/models').read()[:200])"` prints the model list.

5. Prove the environment is contained

```
scripts/check_containment.sh
```

Check: it ends with `PASS contained`. Each service it checks must first reach an internal service (proving the probe works) and then fail to reach the internet. It also checks that `granite-gateway` is the only container on `llm-egress`. If it says FAIL, stop and fix it before loading real policies. See security.md.

6. Load policies

Policy files can be `.md`, `.txt`, `.docx` or `.pdf`. A short header block at the top of each file gives the details the system can't guess:

```
---
id: SAF-112
title: Maintenance Access
owner: Maintenance Manager
parent: POL-007
subsets: [Safety]
version: 1
---
```

Without a header, the ID is taken from the file name or the first lines, and the version counts up. Groups (`subsets`) must be in the header.

```
docker compose run --rm policy-api python -m app.ingest /policies
# Also use the extract model to find references the regex misses (slower):
docker compose run --rm policy-api python -m app.ingest /policies --llm-refs
```

Check: it prints `ingested <ID> v<N>` for each file and lists any skipped files with the reason.

7. Set up Open WebUI

This version needs three things in Open WebUI: the **upload filter** (a global function), the **policy tool**, and the **assistant model** with its system prompt. All three are pasted in by an admin; none is built into the image.

Open `http://<server>:3000`. The first account you create becomes the admin. Add the upload filter first:

1. **Admin Panel → Functions → +**: paste all of `openwebui/upload_filter.py` and save.
2. Open its valves (the gear icon). Set `API_KEY` to `POLICY_API_KEY` from `.env`. Leave `API_URL` as `http://policy-api:8000`.
3. Switch it **on**, and make it **Global** (the toggle in the function's menu). Without Global, attached files are not sent to policy-api, and nobody can decide a paused check from the chat. If you would rather not use Global, enable it on the `Policy Impact Assistant` model only.

See also ui.md. Then:

1. **Turn off sign-up.** Set `OPENWEBUI_ENABLE_SIGNUP=false` in `.env` and run `docker compose up -d open-webui`. Add users from the Admin Panel.
2. **Add the tool.** Workspace → Tools → + → paste all of `openwebui/policy_impact_tool.py` → Save.
3. **Set the tool's settings (Valves).** Open the tool's gear icon. Set `API_KEY` to the `POLICY_API_KEY` from `.env`. Leave `API_URL` as `http://policy-api:8000`.
4. **Create the assistant model.** Workspace → Models → +: - Name: `Policy Impact Assistant`; base model: `granite-4.2-30b` (or your `GRANITE_MODEL`) - System prompt: paste `openwebui/system_prompt.md` - Tools: tick **Policy Impact Checker** - Advanced Params → **Function Calling: Native** - Save, and make it the default model for users (Admin Panel → Settings → Models).
5. **Hide the bare `granite-4.2-30b` model** from regular users, so everyone uses the assistant.

Menu names move between Open WebUI releases. If a step doesn't match, look for the same setting nearby. The Open WebUI tool reads attached files from `__files__`; if uploads stop working after an Open WebUI upgrade, check that first (operations.md).

Check: in a new chat, ask *"Which Safety policies are affected by the change to POL-007?"* and attach the new version. You get an answer with a download link, and the link opens an Excel file.

Turn on the review pause (optional)

By default, "maybe" sections are listed in the answer and the Excel file for someone to check later. To make a check **wait** until a person decides them in chat:

1. Set `REVIEW_GATE=1` in `.env`.
2. Run `docker compose up -d policy-api`.
3. Make sure the upload filter is on and Global (step 7 above). It is the only thing that can record a decision.

Then, for a check that has "maybe" sections, the assistant lists them and shows the check number (8 characters). The person types a message that starts with `decide`, for example `decide 3f9a1c2e SAF-150 3.1 yes, SAF-151 2.4 no`. See how-to.md.

Check: run a check that you know has a "maybe" (with the real models, `SAF-150` in the sample data may be one). The chat shows the list instead of the final answer. Type `decide <check number> all no`. The assistant says it was recorded and then shows the final answer. In the database, `SELECT status FROM runs ORDER BY created_at DESC LIMIT 1;` goes from `needs_review` to `done`.

What is stored for a paused check

Nothing waits in memory. A paused check is a `runs` row with status `needs_review` and a saved checkpoint in Postgres. It survives restarts of any container. Back up the database as usual (operations.md); the checkpoint tables are part of it.

8. Before real use

- [ ] `scripts/check_containment.sh` passes
- [ ] Evaluation on at least 10 past changes meets your recall target (evaluation.md)
- [ ] Security checklist done (security.md)
- [ ] Two or three staff have tried it using only how-to.md, without help

Chapter 4

Operations

Run it day to day, upgrade it, fix problems.

For: Administrators and engineers

Day-to-day running, upgrades, backups and troubleshooting.

Everyday commands

```
docker compose ps                                  # what's running and healthy
docker compose logs -f --tail=100 policy-api       # follow one service
docker compose restart policy-api                  # after editing .env
docker compose stop && docker compose up -d        # full restart
scripts/check_containment.sh                       # after any change
```

Optional services need their profile: `docker compose --profile escalate up -d escalate`.

Health

Check	Healthy looks like
`curl -s http://127.0.0.1:8100/health`	`{"ok":true,"demo_mode":false}` (if `demo_mode` is true, real models are **not** in use)
`docker compose ps`	`db`, `policy-api` and `granite-gateway` show `healthy`
`curl -s $GRANITE_URL/v1/models`	Lists `granite-4.2-30b`. Granite runs on its own server, so check it there (and with whoever runs it), not with `nvidia-smi` here
Recent runs	`SELECT status, count(*) FROM runs WHERE created_at > now() - interval '1 day' GROUP BY 1;` shows few `failed`
Job queue	`curl -s -H "Authorization: Bearer $POLICY_API_KEY" http://policy-api:8000/queue` from inside the network: `oldest_wait_s` near 0 and `dead` not rising

The job queue

Every check is a job in the `jobs` table. `policy-api` runs a worker that takes jobs one at a time per free slot (`WORKER_CONCURRENCY`, default 2) and renews a lease on each while it works (`JOB_LEASE_SECONDS`, default 60).

- **Restarts and crashes don't lose checks.** If `policy-api` restarts, it takes back its own jobs at once. If it dies or the server reboots, the lease runs out and the job is picked up again. The check **carries on after the last finished step**, using its saved checkpoint; only the step that was running is repeated. People waiting in Open WebUI see "Starting again after an interruption…". See Resume after a crash.

- **Temporary problems are retried** (a model server restarting, the database briefly away), up to `JOB_MAX_ATTEMPTS` (default 3) with growing waits. After that the job is `dead` and the person sees "failed several times in a row".
- **Problems with the question itself are not retried** (no such policy, nothing changed).
- **More capacity:** add workers with `docker compose run -d --name worker-2 policy-api python -m app.worker`, or raise `WORKER_CONCURRENCY`. Workers never take the same job twice. Set `WORKER_ENABLED=0` on `policy-api` if you want only separate workers to run checks.

A check that is waiting for a person (`REVIEW_GATE=1`) is **not** a running job. It has finished its job and sits in `runs` as `needs_review`. See Find a paused check.

Dead jobs keep their last error:

```
SELECT id, kind, payload, attempts, last_error, finished_at FROM jobs WHERE status = 'dead' ORDER
BY id DESC LIMIT 20;
-- Retry one after fixing the cause:
UPDATE jobs SET status = 'pending', attempts = 0, run_at = now() WHERE id = <id>;
UPDATE runs SET status = 'queued', error = NULL, finished_at = NULL WHERE id = '<payload
run_id>';
-- The retry carries on from the last saved step. To start the check again from the beginning
instead,
-- delete its checkpoints first (see "Start a check again from scratch" below).
-- Housekeeping (monthly):
DELETE FROM jobs WHERE status = 'done' AND finished_at < now() - interval '30 days';
```

Find a paused check

With `REVIEW_GATE=1`, a check that has "maybe" sections stops and waits. In the chat it shows as a list with a check number. To see every paused check:

```
SELECT left(id::text, 8) AS check_number, head_policy, subset, requested_by, created_at,
       jsonb_array_length(jsonb_path_query_array(result, '$.rows[*] ? (@.status == "Maybe")')) AS
waiting
FROM runs WHERE status = 'needs_review' ORDER BY created_at;
```

Or ask for one check, from inside the network (the short number works on this route):

```
curl -s -H "Authorization: Bearer $POLICY_API_KEY" http://policy-api:8000/runs/3f9a1c2e
# "status": "needs_review", and "review": the sections still waiting
```

A paused check uses no worker and no model, and it waits for ever. Nothing times it out. To see one in the chat, ask the assistant for the result of that check number; it shows the list again, and the person types `decide <check number> ...`. To answer from the command line instead (for example for someone who left), send the same request the filter sends:

```
curl -s -X POST -H "Authorization: Bearer $POLICY_API_KEY" -H "Content-Type: application/json" \
  -d '{"decisions":[{"policy_id":"SAF-150","clause":"3.1","affected":true}],"reviewed_by":"Your
Name"}' \
  http://policy-api:8000/runs/3f9a1c2e/review
```

It answers 202. Use your own name in `reviewed_by`: it is stored in the result and shown as "Confirmed by". Sections you leave out stay "maybe". If you would rather drop the pause for a check nobody will decide, there is no button for it. Leaving it is harmless: it uses nothing.

Resume after a crash

Usually you do nothing. After a crash, a reboot or `docker compose restart policy-api`:

1. A restarted worker takes its own jobs back at once. After a hard crash, the lease (60 seconds) runs out and any worker takes the job.
2. The graph reads its checkpoint and continues from the first step that did not finish.
3. The person waiting sees "Starting again after an interruption…", then the answer.

To check, look at the queue and at the run:

```
SELECT id, kind, status, attempts, run_at, last_error FROM jobs WHERE status IN ('pending',
'running') ORDER BY id;
SELECT status, progress FROM runs WHERE id = '<run id>';
SELECT checkpoint_id, metadata->>'step' AS step FROM checkpoints WHERE thread_id = '<run id>'
ORDER BY checkpoint_id;
```

The last query lists the saved steps for the check. The thread id is the run id. If a check stays stuck after `policy-api` is healthy again, the job is probably `dead`; use the retry SQL under the job queue.

Start a check again from scratch

If a check's saved state is bad (for example after restoring an older database backup, or the same error repeats after a fix), delete its checkpoints, then queue it again:

```
DELETE FROM checkpoint_writes WHERE thread_id = '<run id>';
DELETE FROM checkpoint_blobs  WHERE thread_id = '<run id>';
DELETE FROM checkpoints       WHERE thread_id = '<run id>';
```

Then ask the question again in the chat. That makes a new check with a new number. The checkpoint tables are made by `policy-api` on start; do not create or change them by hand.

How long a run takes

Roughly: number of changed sections × about 20–40 related clauses × time per verdict ÷ `JUDGE_CONCURRENCY`. For example, 10 changed sections with 30 clauses each is 300 verdicts; at 16 at a time and about 3 s each, that's about one minute. Raise `JUDGE_CONCURRENCY` if the Granite server isn't busy; lower it if verdicts time out.

Backups

Back up the database daily. The models and the policy source files are recoverable from their original sources; runs and verdicts are not.

```
docker compose exec -T db pg_dump -U policy -Fc policy > backups/policy-$(date +%F).dump
# Restore into an empty database:
docker compose exec -T db pg_restore -U policy -d policy --clean --if-exists < backups/policy-
YYYY-MM-DD.dump
```

The checkpoint tables are in the same database, so this backs them up too. Restoring an older dump brings back older checkpoints; a check that finished after the dump has no checkpoint, and one that was paused may need to be asked again.

Test a restore every quarter on a spare machine. Also back up the `openwebui` volume (users and the tool configuration), and keep `.env` in your secrets vault.

Upgrades

Change one thing at a time, and run the evaluation after anything that touches the models.

1. Read the release notes for the component.
2. Bring the new image or weights in as in setup.md; record and check hashes.
3. Change the tag in `.env` (or the folder in `compose.yaml`).
4. `docker compose up -d <service>`
5. `scripts/check_containment.sh`
6. Smoke test (testing.md), and evaluate if a model changed.
7. Roll back by restoring the old tag and running `docker compose up -d <service>`.

Granite upgrades happen on the Granite server, not here. After any change there (vLLM version, flags, model files), run `curl -s $GRANITE_URL/v1/models`, then the evaluation. To change the key or address, edit `GRANITE_API_KEY` / `GRANITE_URL` in `.env` and run `docker compose up -d granite-gateway`.

Open WebUI upgrades can change the tool API. Afterwards, attach a file and ask a question. If the tool says no file was attached, check how the new release passes `__files__` and adjust `_attached_file` in `openwebui/policy_impact_tool.py`.

policy-api upgrades: rebuild outside the environment with a new `APP_VERSION`, carry the image in, and set `APP_VERSION` in `.env`. Checks in progress during the restart are picked up again automatically. Database changes in `policy_api/app/migrations/` are applied on start, once each.

Changing the embedding model

The `clauses.embedding` column has a fixed size (768). To switch to a model with a different size:

```
docker compose exec db psql -U policy -c "DROP INDEX clauses_embedding_idx;
  ALTER TABLE clauses ALTER COLUMN embedding TYPE vector(<new size>) USING NULL;
  CREATE INDEX clauses_embedding_idx ON clauses USING hnsw (embedding vector_cosine_ops);"
# Put the new model in models/embedding, restart embed, then re-ingest every policy:
docker compose up -d embed && docker compose run --rm policy-api python -m app.ingest /policies
```

Update `vector(768)` in `db/init.sql` too, so new installs match. Re-run the evaluation.

Moving a model to different GPUs

Run `python3 scripts/setup_gpus.py` again and give the new GPU numbers. Listing several GPUs for `escalate` (or another vLLM model run by this stack) spreads that model across them (tensor parallelism); the script sets `ESCALATE_TP` and the other size settings to match. Then `docker compose up -d escalate` (or whichever service moved). Granite's GPUs are managed on its own server.

Troubleshooting

Symptom	Likely cause	Fix
policy-api exits with `Missing settings`	`.env` incomplete	Fill in the named variables
policy-api exits with `at least 32 characters`	Short key	Regenerate with `openssl rand -hex 32`
`400 Invalid host header`	The Host name isn't in `ALLOWED_HOSTS`	Add the name the proxy sends (`policy-api`) or your hostname
Chat says "The policy service could not be reached"	policy-api down, or the tool's `API_URL` / `API_KEY` Valve is wrong	`docker compose ps` ; check the Valves
Chat says "checking model is not responding"	The Granite server is down or overloaded, or `granite-gateway` cannot reach it	`docker compose logs granite-gateway` ; run `curl -s $GRANITE_URL/v1/models` ; lower `JUDGE_CONCURRENCY`
Many "could not be read" verdicts	Structured output not working with this vLLM version	Check the Granite server's vLLM version supports `response_format` JSON schema; check its logs
The assistant answers without calling the tool	Function calling not set to Native, or the tool isn't enabled on the model	setup.md step 4
"No numbered sections were found" for a Word file	Word automatic numbering	Save with typed numbers, or as PDF (how-to.md)
Download link gives 403	Link expired (72 h) or copied incompletely	Ask the assistant again
Download link gives a connection error	`PUBLIC_BASE_URL` isn't reachable from users' browsers	Set it to the proxy's public address and restart policy-api
A run stays "running"	It's large, or Granite is slow	Check `progress` on the run; the tool stops waiting after 30 minutes but the run continues
A check says it needs a decision, and nobody answers	`REVIEW_GATE=1` and nobody typed `decide …`	Find a paused check
Typing `decide …` does nothing, or the assistant shows a hint	The upload filter is off or not Global, the check number is wrong, or a section is not on the list	setup.md step 3; copy the number from the assistant's message; use only the listed sections
"This check is not waiting for a decision"	Someone else already answered it, or it has finished	Ask the assistant for the result of that check number
Ingest skips files	No ID, no numbered sections, or unreadable type	The skip reason is printed for each file

Symptom	Likely cause	Fix
`check_containment.sh` fails	A service can reach the internet	security.md

Chapter 5

Security and safety practices

Keep policy text contained and the system safe.

For: Everyone

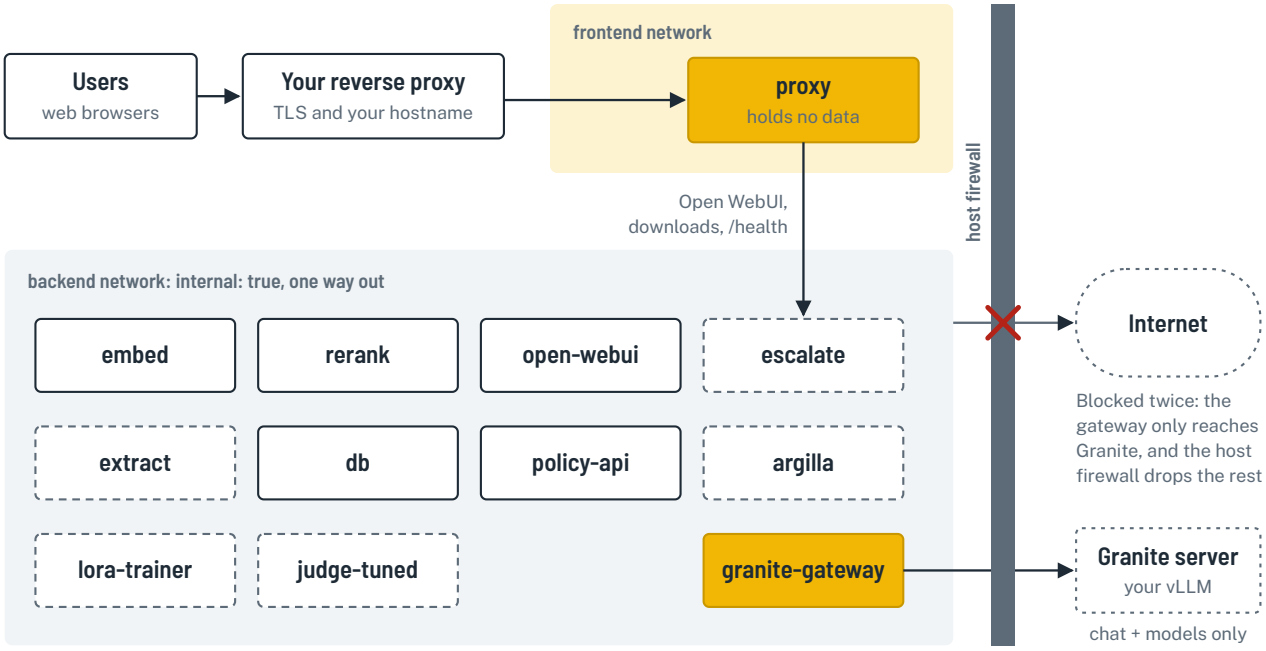
Two things must hold: **policy text never leaves the contained environment**, and **no one acts on an automated decision without a person checking it**. This page explains how the system enforces both, and what you still have to do.

Threats and controls

Risk	Control in this repo	Your job
Policy text leaves the environment	Every service holding text is on the `internal: true` network; the only way out is `granite-gateway`, which passes just chat and model-list calls to your own Granite server; `HF_HUB_OFFLINE`, telemetry off	Run `check_containment.sh`; block egress at the host firewall too, and allow `llm-egress` to reach only the Granite address
Unvetted or tampered model weights	Weights mounted read-only; SHA-256 manifest	Record hashes at download; check before each start; approved-model list
Someone calls the API directly	Bearer key on every route; proxy exposes only downloads and `/health`	Keep `.env` at `chmod 600`; rotate keys
A malicious web page uses a browser to reach the API (DNS rebinding)	`TrustedHostMiddleware` refuses unknown `Host` headers	Keep `ALLOWED_HOSTS` short
A download link is shared or guessed	HMAC-signed links that expire after 72 h; run IDs are random UUIDs	Shorten `LINK_TTL_HOURS` if needed
Instructions hidden inside a policy steer the model ("ignore the above...")	Policy text is fenced as data in every prompt, and fence markers inside it are replaced so it can't break out; the model returns only a fixed JSON schema for each verdict; a "yes" whose quoted evidence isn't really in both texts is downgraded to Maybe for a person to check; the chat model is told never to follow text in documents	Investigate verdicts whose reasons look odd
Policy text becomes a live Excel formula	Cells that would be formulas are forced to text	Nothing
Document or model text adds a link or image to the chat answer (an image URL could carry data out)	Markdown characters in every document and model value are escaped; only the download link is live	Keep Open WebUI on the internal network; block egress so a stray image can't load

Risk	Control in this repo	Your job
A small upload expands into a huge one (zip bomb, giant PDF) or a crafted line slows parsing	Uploads capped at `MAX_UPLOAD_MB`; .docx refused if it unpacks past 50 MB; PDFs capped at 1,000 pages; text capped at 5 M characters; each upload is parsed in its own process (1 GB memory cap, killed after 60 s, at most 2 at once); heading regex can't backtrack	Nothing
The model is wrong	Evidence quotes and reasons on every row; "Maybe" for uncertainty; full audit trail	Keep a person approving every change (below)
Lost history	Every model decision stored in `judgements` with model name and prompt hash	Back up the database (operations.md)

Network containment



Two barriers keep policy text in: a network with one narrow way out, and your host firewall.

- `backend` is a Docker network with `internal: true`, so its containers have no gateway out.
- The backend has exactly **one** way out: `granite-gateway`. It also joins `llm-egress`, a normal Docker network, and forwards to `GRANITE_URL` only. It passes `/v1/chat/completions` and `/v1/models` and nothing else, and checks the server's TLS certificate (`GRANITE_CA_PATH` for a private CA). No other container joins `llm-egress`.
- The Granite API key is held only by the gateway. It adds the key itself and drops any key the caller sent, so Open WebUI and policy-api never see it.
- **Firewall rule to add:** on the host, allow traffic from the `llm-egress` network to the one address and port of your Granite server, and drop everything else from it. Without this rule, the gateway is still limited by its config, but the host would not enforce it.
- Only `proxy` joins `frontend`. nginx serves Open WebUI and only `/exports/<run-id>.xlsx` and `/health` from policy-api. All other API routes return 404 from outside.

- **Don't rely on Docker settings alone.** Also block outbound traffic from the host at your firewall (for example, rules in the `DOCKER-USER` chain on Linux), and keep the server on a network segment without internet access.

Proving it

```
scripts/check_containment.sh
```

The script first checks that `granite-gateway` is the only container on `llm-egress`. Then, for each running service, the script first connects to an internal service. That's the positive control: it proves the probe itself works. Then it tries to reach the internet, and that must fail. The script also fails if it checked nothing. Run it after setup, after every upgrade, and after any network change. Keep the output with your change records.

During development, a network setting meant to block outbound traffic ("no outbound NAT") was ignored by Docker Desktop. The script caught it. That's why containment relies on `internal: true` networks, not driver options.

Secrets

- Every secret lives in `.env` (git-ignored). Generate each with `openssl rand -hex 32`. policy-api refuses to start if its keys are shorter than 32 characters.
- `chmod 600 .env`. Only administrators can read the server.
- **Rotate** by changing the value and restarting the affected services:
- `POLICY_API_KEY`: restart `policy-api` and update the tool's Valve in Open WebUI
- `LINK_SIGNING_KEY`: restart `policy-api`; existing download links stop working
- `LLM_API_KEY`: restart the local model services (escalate, extract), `policy-api` and `open-webui`
- `GRANITE_API_KEY`: change it on the Granite server and in `.env`, then `docker compose up -d granite-gateway`. Nothing else needs it
- Never paste secrets into chats, tickets or a wiki.

Models and supply chain

- Use only models on your organisation's approved list. All defaults are US-developed and published under permissive licenses. Check each model card and license before use.
- Granite 4.2 30B runs on your own vLLM server, outside this stack. Its weights, vLLM version and flags are that server's owner's job. Prefer the built-in `--reasoning-parser nemotron_v3` over the model card's `granite_thinking_parser.py` plugin, which runs code from the model download.
- Download weights outside the environment, record hashes (`scripts/verify_models.sh record`), carry them in, and check them (`verify_models.sh check`) before every start and upgrade.
- Pin every image tag in `.env`. Never use `latest`. Build the policy-api image from a pinned dependency set or your internal mirror.
- vLLM's `--trust-remote-code` (needed for Inkling) runs Python shipped with the model. Review that code and verify the hashes before enabling `escalate`.

Access

- Open WebUI accounts are created by an admin. Sign-up is off after the first admin.
- Everyone with an account can query every group. If some groups must be restricted, run a separate stack per audience, or add per-user group filtering to the tool before go-live.
- The tool sends the asking user's email or name as `requested_by`, which is stored on the run. Tell users their questions are logged.

Human oversight

The system **finds and explains**. It does not approve changes.

- Every "Yes" and "Maybe" goes to a named person, who records a decision (the Review decision column, later the review queue).
- "Maybe" never means "fine". Not-affected policies are listed, with the reason, so reviewers can spot-check them.
- Don't let the tool's output become the record of decision. The decision belongs in your document control system.
- Measure accuracy on real past changes before launch and every quarter (evaluation.md).

Deciding "maybe" sections in chat (`REVIEW_GATE=1`)

The AI model cannot decide. A person types `decide <check number> ...` as their whole message; the Open WebUI upload filter (trusted code) reads it and sends it to policy-api. The check number is the first 8 characters of the check's id, which is only 32 bits. Plainly, what that means:

- It is a convenience, not a password. Someone who guessed or saw a code could decide that check.
- Only signed-in staff can enter it, and only through the chat (the filter holds the service key; people and the model do not).
- It is accepted only as exactly 8 characters (or the full id), and only on the status and review routes. Files, downloads and every other route need the full id.
- For a review it is looked up only among checks that are paused waiting for a decision. A finished or running check cannot be decided, and a code that matches two paused checks is refused.
- Every decision is recorded under the Open WebUI name of whoever typed it, so a wrong or unauthorised one is visible and can be traced.
- The check is not tied to the person who started it. The tool stores the asking person's email or display name as `requested_by`, but that is a label, not a stable user id, so it cannot be matched safely to whoever types the decision. Treat check numbers as internal: do not paste them where people outside the team can see.

Data retention

Uploaded files, runs, verdicts and Excel files stay in the database and the `exports` volume until you delete them. Agree on a retention period with records management. To purge runs older than a set age:

```
-- First, the saved graph state of those runs (it is keyed by run id and is not removed with the
run):
DELETE FROM checkpoint_writes WHERE thread_id IN (SELECT id::text FROM runs WHERE created_at <
now() - interval '2 years');
DELETE FROM checkpoint_blobs  WHERE thread_id IN (SELECT id::text FROM runs WHERE created_at <
now() - interval '2 years');
DELETE FROM checkpoints       WHERE thread_id IN (SELECT id::text FROM runs WHERE created_at <
now() - interval '2 years');
DELETE FROM runs WHERE created_at < now() - interval '2 years';   -- cascades to changes and
judgements
DELETE FROM uploads u WHERE NOT EXISTS (SELECT 1 FROM runs r WHERE r.upload_id = u.id)
                        AND u.created_at < now() - interval '2 years';
```

Then delete the matching `<run-id>.xlsx` files from the `exports` volume.

If something goes wrong

Situation	Do this
`check_containment.sh` fails	Stop `open-webui` and `policy-api` (`docker compose stop`). Fix the network. Re-run the check. Record what happened
A secret may have leaked	Rotate it now (see Secrets). Check the proxy logs for unusual downloads
A wrong result was acted on	Find the run ID on the Excel Summary sheet; pull its `judgements`; add the case to the evaluation set
A policy contains text that tries to instruct the model	Keep the document for analysis; check the verdicts for that policy by hand

Go-live checklist

- [ ] `check_containment.sh` passes and the output is filed
- [ ] Host firewall blocks outbound traffic from the server, except `llm-egress` to the Granite address only
- [ ] `GRANITE_URL` uses `https://` (unless Granite is on this machine) and the certificate check works
- [ ] `.env` secrets generated fresh, file is `chmod 600`, not in git
- [ ] Model hashes verified; models on the approved list
- [ ] Image tags pinned; none is `latest`
- [ ] Open WebUI sign-up turned off; users created by an admin
- [ ] TLS on the reverse proxy in front of `proxy`; `PUBLIC_BASE_URL` uses `https://`
- [ ] Evaluation recall meets the agreed target on a hold-out set
- [ ] Review process for Yes/Maybe agreed with the policy owners
- [ ] Database backups running and restore tested
- [ ] Retention period agreed and written down

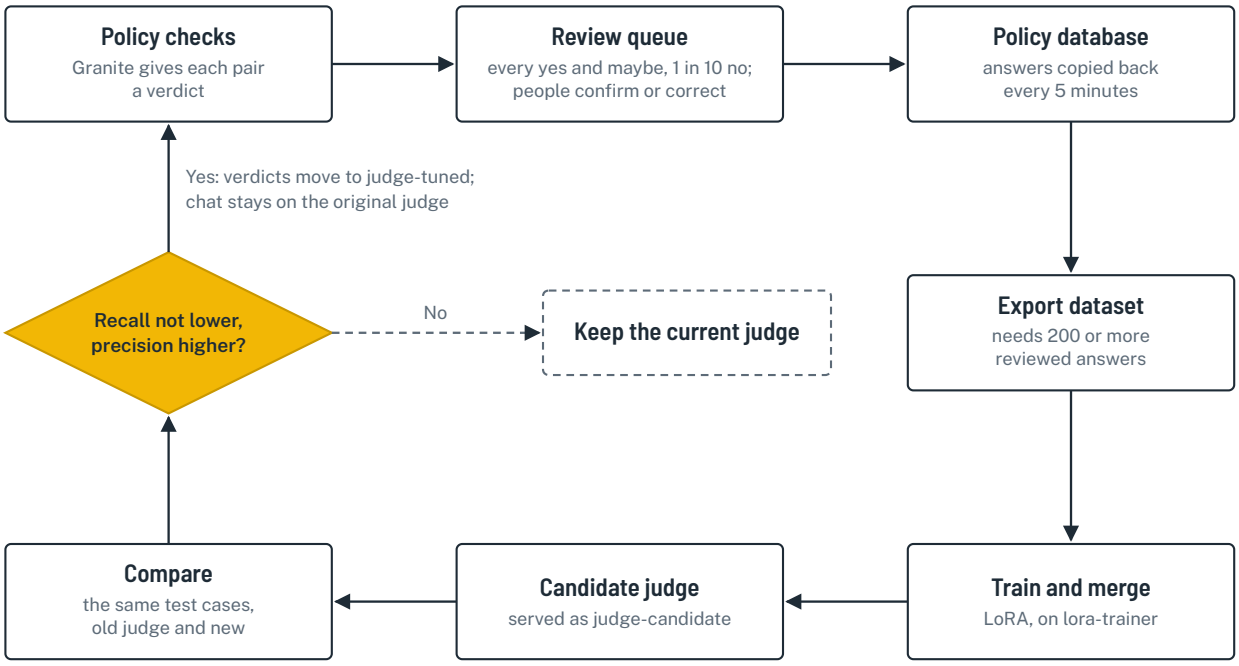
Chapter 6

Review queue and judge training

Let people check answers, then teach the judge from them.

For: Administrators, reviewers and engineers

People check the judge's answers in a review queue (Argilla). Their corrections are saved and, once there are enough, used to fine-tune the judge. Nothing leaves your servers: everything runs on the sealed `backend` network, with telemetry and internet access turned off.



The review and training loop. A new judge replaces the old one only if it scores better.

For the administrator

What you get

(In this version a person can also decide "maybe" sections in the chat when `REVIEW_GATE=1`; see how-to.md. Those decisions change the final answer. They are not copied into the review queue or the training data. The queue below is where training data comes from.)

- **Review queue**: one card per "this section may be affected" answer. The reviewer sees the old and new wording, the other policy's section and the program's answer, and presses Submit.
- **Training**: after a few hundred reviewed cards, an engineer can teach the judge from them. A new judge only replaces the old one if it scores better on your test cases.

Start the review queue

1. In `.env`, fill in the "Review queue (Argilla)" block. Make each secret with `openssl rand -hex 16` (use letters and digits only).
2. Start it: `docker compose --profile review up -d`. The first start takes a few minutes.
3. Ask whoever runs the web proxy to add the Argilla address (see "Engineer: proxy" below).
4. Open `http://<your-server>:6900` and sign in as the admin user from `.env`.

Add a reviewer

Run once per person (change the name, username and password):

```
docker compose exec argilla python -m argilla_server database users create \
  --first-name "Ana" --username ana --password "a-long-password" --role annotator --workspace
policy-review
```

Give them the address, their username and password. A reviewer only sees their own cards. To add a reviewer who can also see everyone's work, use `--role admin` instead of `annotator`.

What reviewers do

Reviewers need no training. The first screen shows the instructions in plain words. For each card:

1. Read the old and new wording, then the section that may be affected.
2. The program's answer is already filled in. If it is right, press **Submit**.
3. If it is wrong, change the answer, pick the kind of impact, add a short note saying why, and press **Submit**.
4. **Can't tell** is allowed. **Discard** skips a card. Nothing is ever lost or published.

Ask reviewers to correct the program when it is wrong, not to agree by default. Corrections are the most useful part.

How answers come back

The sync job in policy-api sends new answers to the queue and copies submitted answers back into the policy database (`reviewer_decision`, `reviewer_impact_type`, `reviewer_note`, `reviewed_by`, `reviewed_at`). A card that is only a draft or was discarded is ignored.

Turn the sync on by setting `ARGILLA_URL=http://argilla:6900` in `.env` (with `ARGILLA_API_KEY` already set) and restarting policy-api. It runs every 5 minutes (`REVIEW_SYNC_SECONDS`) as a job in the job queue. It sends every "yes" and "maybe" from finished checks, plus 1 in 10 of the "no" answers (`REVIEW_SAMPLE_NO`), because reviewers also need to catch what the program missed. Each card shows the clause exactly as the program saw it, even if the policy has changed since.

When is there enough?

Training refuses to start below **200 reviewed answers** and tells you how many more it needs. More is better: aim for 500 or more, covering at least 5 different changed policies, and include plenty of corrections, not only agreements. Check the count any time:

```
docker compose --profile train run --rm lora-trainer export_dataset.py --out /out/data
```

It prints the counts and writes nothing until there is enough.

Back up and delete

The queue's data lives in the `argilla-pgdata`, `argilla-esdata` and `argilla-data` volumes. Back up `argilla-pgdata` with the policy database. The index (`argilla-esdata`) can be rebuilt. Reviewer answers are also copied into the policy database, which is the record that counts.

Engineer

Services (all on `backend`, none publish a port)

Service	What	Profile
`argilla`, `argilla-worker`	Argilla v2.8.0 server and background worker (UI port 6900 inside the network)	review
`argilla-db`	Postgres 16.6, Argilla's own data, kept apart from the policy database	review
`argilla-es`	Elasticsearch 8.17.0, search index	review
`argilla-redis`	Redis 7.4.2, job queue	review
`lora-trainer`	exports data, trains and merges (image built from `training/Dockerfile`)	train
`judge-candidate`	vLLM serving the merged model as `judge-candidate`	train

Versions are set in `.env` (`ARGILLA_TAG`, `ARGILLA_ES_TAG`, `ARGILLA_REDIS_TAG`, `ARGILLA_PG_TAG`). The Python client `argilla==2.8.0` in `policy_api/requirements.txt` must match `ARGILLA_TAG`. Telemetry is off: `HF_HUB_DISABLE_TELEMETRY=1`, `HF_HUB_OFFLINE=1`, `ARGILLA_ENABLE_TELEMETRY=0`.

`argilla` refuses to start if `ARGILLA_ADMIN_PASSWORD` or `ARGILLA_API_KEY` is empty, and the password must be 8+ characters. The first start creates the owner account and workspace from `.env`. Restarting is safe (the account is not created twice). Its healthcheck uses `/api/docs`: in v2.8.0 `/api/v1/status` returns 422 even when healthy.

Proxy: expose the Argilla UI

Reviewers reach Argilla through the existing proxy. Add this `server` block to `proxy/nginx.conf` (next to the Open WebUI one), and publish the port on the `proxy` service: `- "${BIND_ADDR:-127.0.0.1}:6900:6900"`.

```
# Argilla (review queue)
server {
  listen 6900;
  client_max_body_size 25m;
  location / {
    set $argilla http://argilla:6900;
    proxy_pass $argilla;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

policy-api talks to Argilla directly on the backend network, never through the proxy: `ArgillaReview(api_url="http://argilla:6900", api_key=ARGILLA_API_KEY, workspace=ARGILLA_WORKSPACE)` (`policy_api/app/review.py`). Calls are synchronous; call them with `asyncio.to_thread`.

What goes in and out

- `push(items)`: one record per judgement, record id = `str(judgements.id)`. Cards already in the queue are skipped, so a reviewer's work is never overwritten. The program's answer is pre-filled.
- `pull_submitted(since=None)`: only responses with status `submitted`. If two people answered one card, the latest wins. `no` always comes back with impact type `none`. Pass the newest `reviewed_at` you have stored as `since` to pull only new answers.
- Training reads the same rows from Postgres: `reviewer_decision` plus `prompt`, so `prompt` must be the exact text the judge saw (`render_pair`).

Train

All commands run from the repo root. Granite runs on its own server, so the GPUs here only need to hold the training job: give `lora-trainer` (and later `judge-candidate`) GPUs in `compose.gpu.yaml`, like the others:

```
  lora-trainer:
    deploy: {resources: {reservations: {devices: [{driver: nvidia, device_ids: ["0"],
capabilities: [gpu]}]}}}
  judge-candidate:
    deploy: {resources: {reservations: {devices: [{driver: nvidia, device_ids: ["1","2"],
capabilities: [gpu]}]}}}
```

```
docker compose --profile train build lora-trainer          # once; needs internet, or build
elsewhere and `docker load`
docker compose --profile train run --rm lora-trainer export_dataset.py --out /out/data
docker compose --profile train run --rm lora-trainer train_lora.py \
    --train /out/data/train.jsonl --eval /out/data/holdout.jsonl --base /models/gpt-oss-20b --out
/out/adapter
docker compose --profile train run --rm lora-trainer merge_adapter.py \
    --base /models/gpt-oss-20b --adapter /out/adapter --out /out/merged
```

Outputs are in `training/out/` (not in git: they contain policy text). `training/README.md` has the settings and the details.

Evaluate and promote

The rule: **switch only if recall is not lower and precision is higher** on the same test cases.

1. Start the candidate: `docker compose --profile train up -d judge-candidate` (waits for the model to load).
2. Run the same cases against both judges:

```
docker compose exec policy-api python -m app.evaluate /eval/cases.json > training/out/base.json
docker compose exec -e JUDGE_URL=http://judge-candidate:8000/v1 -e JUDGE_MODEL=judge-candidate \
    policy-api python -m app.evaluate /eval/cases.json > training/out/candidate.json
python3 training/compare_eval.py training/out/base.json training/out/candidate.json
```

`app.evaluate` already reads `JUDGE_URL` and `JUDGE_MODEL` from the environment, so it needs no change. Keep `eval/cases.json` free of the policies used for training (it is a separate answer key). 3. If `compare_eval.py` prints `RESULT: the new model is better` (exit code 0), switch.

Switch. The tuned model gets its own service, `judge-tuned`. Only the policy checks' verdicts move to it; the chat in Open WebUI stays on Granite, so a model trained only to give verdicts never has to hold a conversation.

```
docker compose --profile train stop judge-candidate
mv training/out/merged models/judge-tuned-2026-11        # any name; keep one folder per version
```

In `.env` (and assign it GPUs with `python3 scripts/setup_gpus.py`, answering for `judge-tuned`):

```
TUNED_JUDGE_DIR=judge-tuned-2026-11
JUDGE_URL=http://judge-tuned:8000/v1
JUDGE_MODEL=judge-tuned
```

```
docker compose --profile tuned up -d judge-tuned      # wait until `docker compose ps judge-
tuned` is healthy
docker compose up -d policy-api
```

Run one real check from Open WebUI. Every verdict records the model that gave it, so the `judgements` table shows which checks used the tuned judge.

Roll back (seconds, no model reload): delete or comment out the `JUDGE_URL` and `JUDGE_MODEL` lines in `.env`, then `docker compose up -d policy-api`. Stop `judge-tuned` when you no longer need it.

The Granite server is never replaced; the tuned model only takes over the verdicts.

Known limits

- The judge is now Granite 4.2 30B, but these scripts still fine-tune gpt-oss-20b. A tuned gpt-oss-20b can still be tried as `judge-candidate` and must beat Granite on the test cases to replace it. Tuning Granite itself needs `train_lora.py` changed for a dense bf16 model (no `Mxfp4Config`, LoRA on attention and MLP layers) and is not done yet.
- vLLM cannot apply LoRA to gpt-oss mixture-of-experts layers, so training is attention-only by default and the adapter is always merged into a normal model folder before serving.
- The merged model is bf16: about 42 GB for 20b and about 240 GB for 120b, so `judge-candidate` needs correspondingly more GPU memory than the shipped 4-bit model. Re-quantizing is optional and not done here.
- Training on reviewer answers teaches the judge their habits, including mistakes. The test cases are the check; never skip the promotion rule.

Chapter 7

Evaluation

Measure whether it finds the right policies.

For: Policy owners and engineers

Testing shows the code works. **Evaluation shows whether it finds the right policies.** Do it before real use, and again after every change of model, prompt, embedding model or search settings.

What to measure

Measure	Meaning	Target to start with
Recall	Of the policies that really needed updating, how many did it flag (Yes or Maybe)?	**≥ 0.95**. Missing one is the costly error
Precision	Of the policies it flagged, how many really needed updating?	≥ 0.5 at first. Each false flag costs a reviewer a few minutes
Candidate recall	Did the truly affected clauses even reach the judge (step 3)?	≥ 0.98. If a clause never reaches the judge, no model can flag it
Maybe rate	Share of flagged policies marked Maybe rather than Yes	Watch it fall as the model improves
Unreadable verdicts	Verdicts with "could not be read"	0

Set your own targets with the policy owners. Write them down before you look at results.

Build the case set

Each case is a **real past change** to a head policy, with the list of policies that people actually had to update afterwards.

```
{
  "name": "POL-007 2025 revision",
  "head_policy": "POL-007",
  "subset": "Safety",
  "new_version_file": "private/POL-007-v4.docx",
  "expected_impacted": ["SAF-112", "SAF-130", "SAF-150"],
  "source": "Change request CR-2025-114; updated policies listed in the closure note"
}
```

Where the answers come from matters more than how many cases you have:

- **Use the record of what was actually updated** (change requests, document control logs, revision histories), not a fresh list someone writes while looking at this system's output.
- **Load the policy library as it was before the change.** Otherwise the affected policies already contain the new wording, and the run finds nothing to flag.

- **Keep a separate hold-out set.** Tune `SEMANTIC_K`, `RERANK_KEEP` and prompts on one set of cases, and report results on a different set you never looked at while tuning. A score on the cases you tuned against tells you little.
- **Include hard cases on purpose:** a renamed defined term, a renumbered section, a removed section, a policy that's affected but never cites the head policy, and a change that affects nothing.
- Start with **10–20 cases**, and grow to 50+ over time. Store them in `eval/private/` and `eval/cases.json`, which git ignores.

Historical records are imperfect: people sometimes missed policies too. When the system flags a policy that the record doesn't list, have the policy owner decide. If it really was affected, fix the case and note it in `source`. This is often the most useful finding.

Run it

```
docker compose exec policy-api python -m app.evaluate /eval/cases.json --min-recall 0.95
```

- By default, Yes **and** Maybe count as found, because both reach a person. `--yes-only` measures how often the model is confident and right.
- The command exits non-zero if recall is below `--min-recall` or any case errored, so it can gate a release.
- The output lists, for every case, which expected policies were **missed** and which **extra** ones were flagged.

Run evaluation against a **test database** loaded with the historical policy versions, never the live one.

Finding why something was missed

For each missed policy, check where it was lost:

```
-- Did any of its clauses reach the judge?
SELECT candidate_number, found_by, stage, result->>'impacted', result->>'why'
FROM judgements WHERE run_id = '<run>' AND candidate_policy = '<missed id>';
```

What you see	Cause	Fix
No rows	It never became a candidate	Check that it's in the group; add explicit references (`--llm-refs`); raise `SEMANTIC_K` / `RERANK_KEEP`; try another embedding model
Rows, all "no"	The judge decided wrongly	Read the pair. Adjust the judge prompt in `app/prompts.py`; turn on escalation; later, fine-tune
Rows, "unsure" turned "no" by escalation	The larger model overruled	Compare both verdicts; decide which model to trust for this kind of change
The changed section wasn't detected	Parsing or diff problem	Check the file's numbering (how-to.md)

Comparing models or settings

Change **one thing at a time** and run the same case set. Record the results in a table like this:

Date	Judge	Escalation	Embed / rerank	K / keep	Recall	Precision	Maybe rate	Time per run

Small case sets are noisy. A difference of one case in 15 is about 7 points of recall. Don't adopt a change on a gain smaller than that unless it holds on the hold-out set.

After launch

- Every reviewer decision on a Maybe is a new labelled example. They are collected through the review queue; see review-and-training.md.
- Each quarter, rerun the evaluation and add new real changes to the case set.
- Before fine-tuning, keep the hold-out cases out of the training data.

Chapter 8

Testing

Run and understand the tests.

For: Engineers

Tests answer one question: **does the code do what it claims?** Whether the *models* find the right policies is a separate question, answered by evaluation.md.

Levels

Level	Command	Needs	Proves
1. Unit + flow	`pytest -q` (in `policy_api/`)	Python only	Parsing, diff, candidate search, verdict handling, chat answer, Excel, API auth and flow
2. Postgres	`TEST_DATABASE_URL=… pytest -q`	Docker	The real store: SQL, pgvector search, versions, reference graph, audit rows, restart handling
3. Stack (demo)	`DEMO_MODE=1` + Compose	Docker	Images build, services start, proxy rules, download links, network containment
4. Stack (real models)	Smoke test below	GPU server	vLLM and TEI wiring, structured output, tool calling in Open WebUI
5. Evaluation	`python -m app.evaluate`	GPU server + labelled cases	Whether it finds the right policies (evaluation.md)

Run levels 1–2 on every change. Run 3 before every release. Run 4–5 after any model, image or prompt change.

Level 1: unit and flow tests

```
cd policy_api
python3 -m venv .venv && .venv/bin/pip install -r requirements-dev.txt
.venv/bin/python -m pytest -q
```

File	Covers
`test_documents.py`	Clause splitting (headings, list items, duplicate numbers), reference and defined-term extraction, front matter, diff (modified, added, removed, renumbered)
`test_pipeline.py`	The full sample run with stand-in models: which policies are Yes/Maybe/not affected, evidence, counts, the chat answer, workbook sheets, **policy text can't become an Excel formula**
`test_graph.py`	The LangGraph run: "unsure" pairs go to the second model only when it is on; **a crash carries on from the last finished step**; the review pause waits until a person decides; review over the API; short check numbers (exactly 8 characters, status and review routes only)

File	Covers
`test_upload_filter.py`	The upload filter: attachments, and typed `decide …` commands (only the person's own message counts; quoted or pasted lines, placeholders and malformed commands record nothing)
`test_api.py`	API key required, **wrong Host header refused** (DNS-rebinding guard), lookups, upload → run → download, tampered download link refused, helpful errors

The stand-in models in `app/fakes.py` are deterministic. The fake judge says "yes" when a linked clause still contains wording the head policy removed, "unsure" when a clause is linked but has no such wording, and "no" otherwise. That makes flow tests exact. It does **not** make them evidence that the real models work.

Level 2: Postgres

```
docker run -d --name sk-test-db -e POSTGRES_PASSWORD=test -p 127.0.0.1:55432:5432 \
  -v "$PWD/../db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro" pgvector/pgvector:pg17
TEST_DATABASE_URL=postgresql://postgres:test@localhost:55432/postgres .venv/bin/python -m pytest
-q
docker rm -f sk-test-db
```

`test_pgstore.py` truncates its tables, and also runs the graph with Postgres checkpoints (a fresh process picks a paused check up from the database), so never point `TEST_DATABASE_URL` at a real database.

Level 3: the stack in demo mode

Uses the sample policies and stand-in models, so no GPUs are needed. Make a throwaway env file (secrets from `openssl rand -hex 32`, any image tags, and `DEMO_MODE=1`), then:

```
docker compose --env-file /tmp/test.env up -d --build db policy-api proxy
curl -s http://127.0.0.1:8100/health                       # {"ok":true,"demo_mode":true}
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8100/subsets   # 404: API routes not
exposed
COMPOSE_ARGS="--env-file /tmp/test.env" scripts/check_containment.sh     # PASS contained
docker compose --env-file /tmp/test.env exec policy-api python -m app.evaluate
/eval/cases.example.json --demo
docker compose --env-file /tmp/test.env down -v
```

Testing the Open WebUI tool without Open WebUI

The tool's logic can be run inside the policy-api container, which has the same libraries:

```
docker cp openwebui/policy_impact_tool.py secretary-keeper-policy-api-1:/tmp/tool.py
docker compose exec -T -e KEY=<POLICY_API_KEY> policy-api python3 - <<'EOF'
import asyncio, os, sys; sys.path.insert(0, "/tmp")
from tool import Tools
t = Tools(); t.valves.API_KEY = os.environ["KEY"]
files = [{"name": "POL-007-v4.md", "file": {"path": "/samples/changes/POL-007-v4.md"}}]
print(asyncio.run(t.check_policy_change("Safety", __files__=files)))
EOF
```

Also try: an unknown group, an ambiguous policy name, no file and no policy, and a wrong `API_URL`. Each must return a plain instruction for the chat model, never a stack trace.

Level 4: smoke test with the real models

On the GPU server, after setup.md:

1. From inside the network, `docker compose exec policy-api python3 -c "import httpx; print(httpx.get('http://granite-gateway:8000/v1/models').json())"` lists `granite-4.2-30b` (the gateway adds the key).
2. Ingest `samples/policies`, upload `samples/changes/POL-007-v4.md` in Open WebUI, and ask about Safety.
3. Expect SAF-112 and SAF-130 flagged, and SAF-141 and SAF-160 not affected. SAF-150 may be Yes or Maybe.
4. In `judgements`, every `result` has all six fields, and none has the "could not be read" reason.
5. Ask something ambiguous ("check the lockout thing") and confirm the assistant asks one question back.

Then ingest a copy of your real policies **in a test database** and run the evaluation.

Testing the tests

A test that can't fail proves nothing. When you add or change a check, break the thing it guards once and confirm the check fails:

- The containment script must fail when a service *can* reach the internet. During development it did: Docker Desktop ignored the "no outbound NAT" network setting, which is why policy-api and Open WebUI now sit only on the sealed `backend` network.
- It must also fail when it checked nothing. An early version printed "PASS" because it couldn't read the env file and skipped every service. It now fails if zero services were checked, and each check starts with a positive control (reaching an internal service).
- The formula-injection test checks the cell's *type*, not just its text.